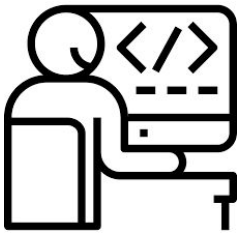


Algoritmos y Programación I

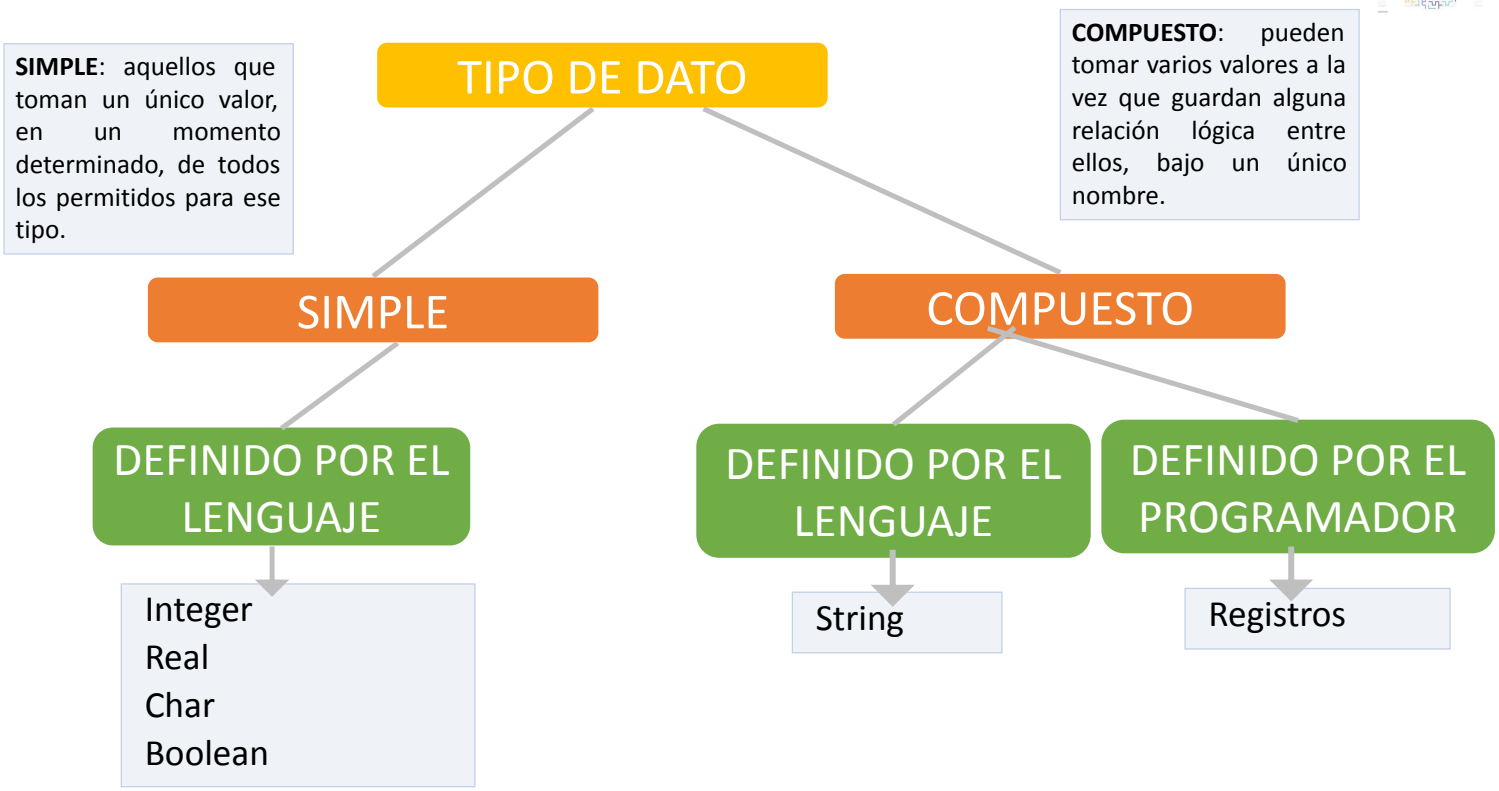
AyPI – **Temas de la clase pasada**

- Funciones
- Alcance de variables

AyPI – **Temas de la clase de hoy**

- Tipo de Dato String
- Tipo de Datos Estructurado
- Clasificación de tipos de Datos Estructurados
- Tipo de datos REGISTRO

AyPI – **TIPOS DE DATOS** ESTRUCTURADOS



AyPI – Tipo de Dato

DATO STRING

Un tipo de dato string es una sucesión de caracteres de longitud fija, que se almacenan en un área contigua de la memoria. Existe en la mayoría de los lenguajes como un tipo predefinido.

Tipo de datos **string**

- Es un tipo de dato compuesto, ordinal
- Los valores son de la forma:
  - ‘algo’
  - ‘Algo’
  - ‘Código: 45GVF&dd#FR’
  - ‘ “Hola mundo!!!!” ’
  - ‘$ 5000’
  - ‘’

AyPI – Tipo de Dato

DATO CARÁCTER

Operaciones

Operador de concatenación	Operadores de comparación
• +	• <, <= • >, >= • = • <>

La Tabla ASCII contiene todos los caracteres y el orden entre los mismos. **http://ascii.cl/es/**

AyPI – **TIPOS DE DATO STRING**

```
Program uno;
var
   nombre: string; apellido: string;
   nombre_y_apellido: string;
begin
  nombre:= ‘Rosa’;   apellido:= ‘Gonzalez’;
  nombre_y_apellido:= apellido + ‘, ’ + nombre;
  write(nombre_y_apellido);

  read(nombre);
  if(nombre = ‘Rosa’) then
     write(‘Hola Rosita, amiga mia’)
  else
     write(‘Hola ’ + nombre);
end.
```

Gonzalez, Rosa

AyPI – **TIPOS DE DATO STRING**

```
Program dos;
var
   dato1, dato2, dato3: string;
begin
   dato1:= ‘Rosa’;
   dato2:= ‘ROSA’;
   dato3:= ‘54.3 %’;

   writeln(dato1 < dato2);
   writeln(dato1 > dato3);
   writeln(dato2 >= dato3);
end.
```

¿Qué imprime?

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** CLASIFICACIÓN

ESTRUCTURA DE DATOS

Permite al programador definir un tipo al que se asocian diferentes datos que tienen valores lógicamente relacionados y asociados bajo un nombre único.

Elementos	Acceso	Tamaño	Linealidad
Homogénea	Secuencial	Dinámica	Lineal
Heterogénea	Directo	Estática	No Lineal

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** CLASIFICACIÓN

ELEMENTOS

Depende si los elementos son del mismo tipo o no.

Homogénea → Los elementos que la componen son del mismo tipo.

Heterogénea → Los elementos que la componen pueden ser de distinto tipo.

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** CLASIFICACIÓN

TAMAÑO

Hace referencia a si la estructura puede variar su tamaño durante la ejecución del programa.

Estática → El tamaño de la estructura no varía durante la ejecución del programa.

Dinámica → El tamaño de la estructura puede variar durante la ejecución del programa.

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** CLASIFICACIÓN

ACCESO

Hace referencia a cómo se pueden acceder a los elementos que la componen.

Secuencial → Para acceder a un elemento particular se debe respetar un orden predeterminado, por ejemplo, pasando por todos los elementos que le preceden, por ese orden.

Directo → Se puede acceder a un elemento particular, directamente, sin necesidad de pasar por los anteriores a él, por ejemplo, referenciando una posición.

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** CLASIFICACIÓN

LINEALIDAD

Hace referencia a cómo se encuentran almacenados los elementos que la componen.

Lineal

Está formada por ninguno, uno o varios elementos que guardan una relación de adyacencia ordenada donde a cada elemento le sigue uno y le precede uno, solamente.

No lineal

Para un elemento dado pueden existir 0, 1 ó más elementos que le suceden y 0, 1 ó mas elementos que le preceden.

AyPI – **Tipos de Datos definidos por el programador**

HASTA AHORA

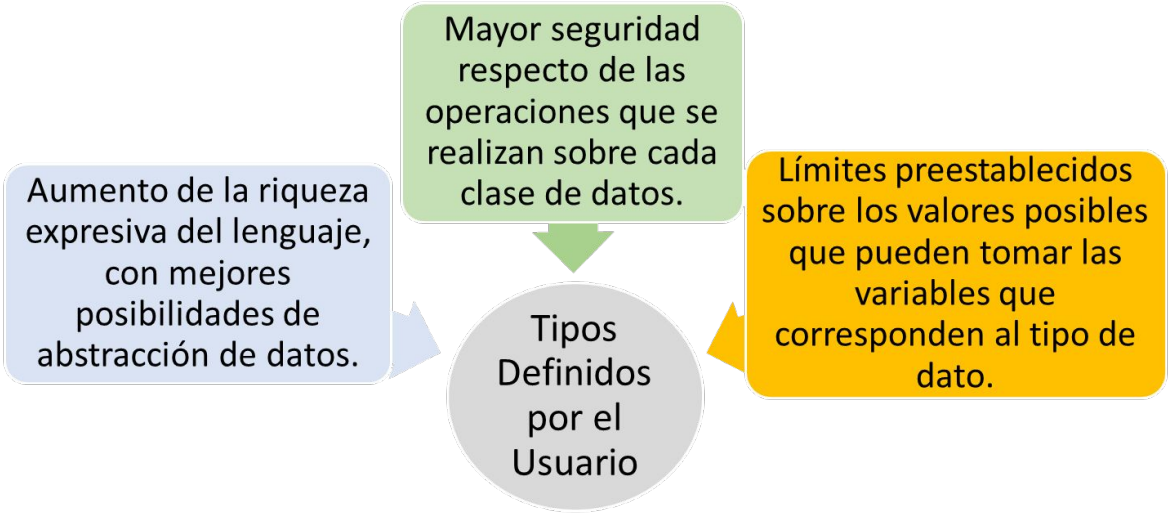
Hemos trabajado los tipos de datos que están incorporados en la mayoría de los lenguajes de programación.

Esto significa que el conjunto de valores de ese tipo, las operaciones que se pueden efectuar y su representación están definidas y acotadas por el lenguaje.

Esta definición nos indica que podemos requerir la representación de elementos "NO de tipo estándar" que se asocien con el fenómeno real a tratar.

AyPI – **Tipos de Datos definidos por el programador**

Algunos aspectos que resultan importantes en los lenguajes de programación es la capacidad de especificar y manejar datos no estándar, pudiendo indicar los valores permitidos, las operaciones válidas y/o su representación interna.



AyPI – **Tipos de Datos definidos por el programador**

TIPO DE DATOS DEFINIDOS POR EL USUARIO

Un tipo de dato definido por el usuario es aquel que no existe en la definición del lenguaje, y el programador es el encargado de su especificación.

Type

```
identificador = ... {Tipo};
```

Dónde se declara?

AyPI – **Tipos de Datos definidos por el programador**

TIPO DE DATOS DEFINIDOS POR EL USUARIO (TDDU)

```
program tddu;
Const
   ...
Type
    identificador = ... {tipo};
{Módulos}
Var
   x: identificador;
   ...
Begin
   ...
End.
```

AyPI – **TIPOS DE DATOS** **ESTRUCTURADOS**

Supongamos que se quiere representar la información de los distintos inmuebles con los que trabaja una inmobiliaria. Para simplificar el problema la inmobiliaria trabaja con casas o departamentos, y sólo los alquila.

¿Qué información debería conocer la inmobiliaria?

AyPI – **TIPOS DE DATOS**

ESTRUCTURADOS

Tipo propiedad

Cantidad de habitaciones

Cantidad de baños

Precio Alquiler

Localidad

Hasta ahora no conocemos una forma de poder agrupar toda esta información en una sola variable.

AyPI – **TIPOS DE DATOS**

ESTRUCTURADOS

Tipo propiedad
Cantidad de habitaciones
Cantidad de baños
Precio Alquiler
Localidad

Tipo propiedad
Cantidad de habitaciones
Cantidad de baños
Precio Alquiler
Localidad

INMUEBLE

AyPI – **TIPOS DE DATOS** ESTRUCTURADOS

ESTRUCTURA DE DATOS

Permite al programador definir un tipo al que se asocian diferentes datos que tienen valores lógicamente relacionados y asociados bajo un nombre único.

Tipo propiedad
Cantidad de habitaciones
Cantidad de baños
Precio Alquiler
Localidad

INMUEBLE

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** REGISTRO

REGISTRO

Es uno de los tipos de datos estructurados, que permiten agrupar diferentes clases de datos en una estructura única bajo un sólo nombre

Tipo propiedad
Cantidad de habitaciones
Cantidad de baños
Precio Alquiler
Localidad

REGISTRO INMUEBLE

Una manera natural y lógica de agrupar los datos de cada inmueble en una sola estructura es declarar un tipo **REGISTRO** asociando el conjunto de datos de cada inmueble.
Un registro está compuesto por **Campos** que representan cada uno de los datos que lo forman

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** REGISTRO

Tipo propiedad
Cantidad de habitaciones
Cantidad de baños
Precio Alquiler
Localidad

REGISTRO INMUEBLE

Heterogénea

Los elementos pueden ser de distinto tipo (puede haber registros con todos elementos del mismo tipo).

Estática

El tamaño no cambia durante la ejecución (se calcula en el momento de compilación).

Directo

Se acceden a cada campo mediante notación puntual.

¿Cómo lo defino?

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** REGISTRO

```
Program registros;
Const
   ....
Type

nombre = record
    campo1: ... {tipo};
    campo2: ... {tipo};
    ...
end;

Var
    variable: nombre;
```

Se nombra cada campo.
Se asigna un tipo a cada campo.

¿Cómo declaro el registro inmueble?

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** REGISTRO

```
Program inmobiliaria;
Const
  ....
Type

inmueble = record
   tipo: string;
   cantHab: integer;
   cantBaños:integer;
   precio: real;
   localidad: string;
end;

Var
   inmu1, inmu2: inmueble;
```

¿Cómo trabajamos el registro inmueble?

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** REGISTRO

CON LA VARIABLE REGISTRO

```
Program inmobiliaria;
Const
  ...
Type

  inmueble = record
    tipo: string;
    cantHab: integer;
    cantBaños:integer;
    precio: real;
    localidad: string;
  end;

Var inmu1, inmu2: inmueble;

Begin
 ...
 inmu2:= inmu1;
 ...
End.
```

La única operación permitida es la asignación entre dos variables del mismo tipo

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** REGISTRO

CON LOS CAMPOS DEL REGISTRO

¿Cómo le damos valor?

```
Program inmobiliaria;
Const
  ...
Type

inmueble = record
   tipo: string;
   cantHab: integer;
   cantBaños:integer;
   precio: real;
   localidad: string;
end;

Var
   inmu1, inmu2: inmueble;
```

```
Begin
 ...
Puedo realizar las
operaciones permitidas
según el tipo de campo
del registro
 ...
End.
```

La única forma de acceder a los campos es variable.nombrecampo

inmu1.cantHab

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** REGISTRO

```
Program inmobiliaria;
Const
  ...
Type

inmueble = record
   tipo: string;
   cantHab: integer;
   cantBaños:integer;
   precio: real;
   localidad: string;
end;

Var
   inmu1, inmu2: inmueble;
```

```
Begin
 inmu1.tipo:='Casa';
 inmu1.cantHab:= 2;
 inmu1.cantBaños:= 1;
 inmu1.precio:= 15.500;
 inmu1.localidad:= 'La Plata';
End.
```

```
Begin
 read(inmu1.tipo);
 read(inmu1.cantHab);
 read(inmu1.cantBaños);
 read(inmu1.precio);
 read(inmu1.localidad);
End.
```

¿Qué ocurre si no le doy valor a todos los campos? Habrá campos con dato "basura".

¿Debo asignarlos en el orden en que se declararon? NO necesariamente.

¿MODULARIZAR?

No se puede hacer read (inmu1)

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** REGISTRO

```
Procedure leer (var i:inmueble);

Begin
    read(i.tipo);
    read(i.cantHab);
    read(i.cantBaños);
    read(i.precio);
    read(i.localidad);
end;
```

¿Debo asignarlos en el orden en que se declararon?

¿Qué ocurre si no le doy valor a todos los campos?

¿Puede ser una función en vez de un procedimiento?

```
Program inmobiliaria;
Const
  ...
Type
inmueble = record
   tipo: string;
   cantHab: integer;
   cantBaños:integer;
   precio: real;
   localidad: string;
end;
Procedure leer (var i:inmueble);
begin
 ...
end;

Var
   inmu1, inmu2: inmueble;
Begin
 leer (inmu1);
 inmu2:= inmu1;
End.
```

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** REGISTRO

```
Procedure leer (var i:inmueble);

Begin
    read(i.tipo);
    read(i.cantHab);
    read(i.cantBaños);
    read(i.precio);
    read(i.localidad);
end;
```

¿Qué ocurre si no le doy valor a todos los campos?

(El campo sin asignar queda con dato basura)

¿Debo asignarlos en el orden en que se declararon?

(no necesariamente)

¿Puede ser una función en vez de un procedimiento?

(Si, la función devuelve un dato tipo inmueble)

¿Cómo muestro el contenido de un registro?

AyPI – TIPOS DE DATOS ESTRUCTURADOS REGISTRO

```
Program inmobiliaria;
Const
  ....
Type

inmueble = record
   tipo: string;
   cantHab: integer;
   cantBaños:integer;
   precio: real;
   localidad: string;
end;

Var
   inmu1, inmu2: inmueble;
```

```
Begin
 leer (inmu1);
 write (inmu1.tipo);
 write (inmu1.cantHab);
 write (inmu1.cantBaños);
 write (inmu1.precio);
 write (inmu1.localidad);
End.
```

No se puede hacer write (inmu1)

¿Qué ocurre si no le imprimo todos los campos?
Nada, no se muestran algunos campos.

¿MODULARIZAR?

AyPI – TIPOS DE DATOS ESTRUCTURADOS REGISTRO

```
Procedure imprimir (i:inmueble);

Begin
   write(i.tipo);
   write(i.cantHab);
   write(i.cantBaños);
   write(i.precio);
   write(i.localidad);
end;
```

¿Debo imprimir en el orden en que se declararon?

¿Qué ocurre si no imprimo todos los campos?

¿Puede ser una función en vez de un procedimiento?

```
Program inmobiliaria;
Const
  ...
Type
inmueble = record
   tipo: string;
   cantHab: integer;
   cantBaños:integer;
   precio: real;
   localidad: string;
end;
Procedure imprimir (i:inmueble);
begin
 ...
end;

Var
   inmu1, inmu2: inmueble;
Begin
 leer (inmu1);
 imprimir(inmu1);
End.
```

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** REGISTRO

```
Procedure imprimir (i:inmueble);

Begin
   write(i.tipo);
   write(i.cantHab);
   write(i.cantBaños);
   write(i.precio);
   write(i.localidad);
end;
```

¿Qué ocurre si no imprimo todos los campos?

(El campo no se muestra por pantalla)

¿Debo imprimir en el orden en que se declararon?

(no necesariamente)

¿Puede ser una función en vez de un procedimiento?

(Terminantemente NO!!!!!)

¿Cómo comparo el contenido de dos registros?

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** REGISTRO

```
Program inmobiliaria;
Const
  ...
Type

inmueble = record
   tipo: string;
   cantHab: integer;
   cantBaños:integer;
   precio: real;
   localidad: string;
end;

Var
   inmu1, inmu2: inmueble;
```

```
Begin
 leer (inmu1);
 leer (inmu2),

 if ((inmu1.tipo = inmu2.tipo) and
     (inmu1.cantHab = inmu2.cantHab) and
     (inmu1.cantBaños = inmu2.cantBaños) and
     (inmu1.precio = inmu2.precio) and
     (inmu1.localidad = inmu2.localidad)) then
  write (`Los registros tienen los mismos valores`)
End.
```

No se puede hacer inmu1 = inmu2

¿MODULARIZAR?

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** REGISTRO

```
function iguales (i1,i2:inmueble):boolean;
Var
 ok:boolean;
Begin
    if ((i1.tipo = i2.tipo)and
     (i1.cantHab = i2.cantHab) and
     (i1.cantBaños = i2.cantBaños) and
     (i1.precio = i2.precio) and
     (i1.localidad = i2.localidad))
    then ok:= true
    else ok:= false;
    iguales:= ok;
end;
```

```
function iguales (i1,i2:inmueble):boolean;
Begin
    iguales:= ((i1.tipo = i2.tipo) and
     (i1.cantHab = i2.cantHab) and
     (i1.cantBaños = i2.cantBaños) and
     (i1.precio = i2.precio) and
     (i1.localidad = i2.localidad));
end;
```

AyPI – **TIPOS DE DATOS ESTRUCTURADOS** REGISTRO

```
Program inmobiliaria;
Const
  ...
Type
inmueble = record
   tipo: string;
   cantHab: integer;
   cantBaños:integer;
   precio: real;
   localidad: string;
end;
function iguales (i1,i2:inmueble): boolean;
begin
 ...
end;

Var
   inmu1, inmu2: inmueble;
```

```
Begin
 leer (inmu1);
 leer (inmu2);
 if (iguales (inmu1,inmu2) = true)
 then write (`Los registros son iguales`)
 else write (`Los registros no son iguales`);
End.
```

```
Begin
 leer (inmu1);
 leer (inmu2);
 if (iguales (inmu1,inmu2))
 then write (`Los registros son iguales`)
 else write (`Los registros no son iguales`);
End.
```

AyPI – **REGISTRO**

EJERCITACIÓN

Escriba un programa que lea inmuebles hasta leer un inmueble cuya localidad es `XXX` Al finalizar informe de los inmuebles en la localidad de `La Plata` cuantos tienen al menos 2 habitaciones

```
Tipo `Depto`
cantHab:2
cantBaños 1
precio 15.200
Localidad La Plata
```

```
Tipo `Casa`
cantHab:3
cantBaños 2
precio 23.000
Localidad Gonnet
```

```
Tipo `Casa`
cantHab:5
cantBaños 3
precio 55.400
Localidad La Plata
```

```
Tipo `Casa`
cantHab:1
cantBaños 2
precio 18.000
Localidad La Plata
```

```
Tipo `Casa`
cantHab:4
cantBaños 1
precio 10.000
Localidad XXX
```

2

Al leer la Localidad XXX, ¿necesito leer todos los otros campos?

NO

AyP – **REGISTRO**

EJERCITACIÓN

Escriba un programa que lea inmuebles hasta leer un inmueble cuya localidad es `XXX` Al finalizar informe de los inmuebles en la localidad de `La Plata` cuantos tienen al menos 2 habitaciones

```
Inicializar contadores (cant)
Leer registro (inmu)
While (no sea el ultimo registro) do
 begin
  if (inmu es de La Plata con al menos dos habitaciones) then
    incremento (cant)
  leer registro (inmu)
 end;
Write (`La cantidad es`, cant);
```

¿Cómo verifico las condiciones?

¿Qué modularizo?

AyPI – **REGISTROS**

EJERCITACIÓN

```
Program inmobiliaria;
Const
  ...
Type
inmueble = record
   tipo: string;
   cantHab: integer;
   cantBaños:integer;
   precio: real;
   localidad: string;
end;

// módulos


Var
   inmu1, inmu2: inmueble;
   cant: integer;
```

```
Begin
 cant:=0;

 leer (inmu);

 while (inmu.localidad <> `XXX`) do
  begin
   if (cumple (inmu1) = true) then
     cant:= cant + 1;
   leer (inmu);
  end;
  write (`La cantidad es`, cant);
End.
```

AyPI – **REGISTROS**

EJERCITACIÓN

```
procedure leer (var i:inmueble);

Begin
   read(i.localidad);
   if (i.localidad <> `XXX`) then
    begin
     read(i.cantHab);
     read(i.cantBaños);
     read(i.precio);
     read(i.tipo);
    end;
end;
```

AyPI – **REGISTROS**

EJERCITACIÓN

```
function cumple(i:inmueble): boolean;
var
 ok:boolean;

begin
 if (i.localidad = `La Plata`) and
    (i.cantHab >= 2) then
  ok:= true
 else
  ok:= false;
 cumple:= ok;
end;
```

Otra opción

```
function cumple(i:inmueble): boolean;
begin
 cumple:= (i.localidad = `La Plata`)
              and (i.cantHab >= 2);
end;
```

AyPI – **Actividad**

Modificar la actividad de la clase 6 incorporando el tipo de datos Terreno. Un Terreno tiene frente, fondo, dirección, localidad y valuación fiscal en dólares.

Las funciones que devuelven el área, la valuación fiscal en pesos y en dólares NO deben ser modificadas.

Modificar el programa para que procese la información de terrenos hasta ingresar la localidad ‘ZZZ’. El programa debe leer el valor del metro cuadrado en pesos y la tasa de cambio a dólares. Luego, para cada terreno se debe leer los valores del frente, fondo, localidad y dirección. El programa debe imprimir todos los datos del terreno con mayor valuación fiscal.